# User Guide

**Clinical Synthetic Data Generation Framework**

A practical guide to generating, evaluating, and comparing synthetic tabular data for healthcare applications.

---

## Table of Contents


- 1. Installation and Setup
- 2. Data Preprocessing and EDA
- 3. Baseline Model Evaluation (Default Parameters)
- 4. Hyperparameter Optimization
- 5. Optimized Model Evaluation
- 6. Generating Synthetic Samples
- 7. SDAC Metrics Reference
- 8. Architecture Reference


---

## 1. Installation and Setup

### System Requirements

| Requirement | Minimum | Recommended |
|---|---|---|
| Python | 3.10+ | 3.10 |
| RAM | 8 GB | 16+ GB |
| Disk | 10 GB | 50 GB |
| GPU | Optional | NVIDIA (CUDA) |

### AWS SageMaker Setup (Recommended)

1. Log in to AWS via Okta > `tec-rnd-sqs-dev`
2. Open **SageMaker AI > Notebook Instances**
3. Create instance: `ml.g4dn.xlarge`, 50 GB volume
4. Add Git repo: `https://github.com/gcicc/tableGenCompare.git` (branch: `main`)
5. Open JupyterLab and run first-time setup:

```
source ~/anaconda3/bin/activate
conda init bash && exec bash
conda create -n tablegen python=3.10 -y
conda activate tablegen
cd ~/SageMaker/tableGenCompare
git submodule update --init --recursive
pip install -r requirements.txt
python -m ipykernel install --user --name tablegen --display-name "Python (tablegen)"
```

1. Select kernel: **Kernel > Change Kernel > Python (tablegen)**

### Local Installation

```
git clone --recurse-submodules https://github.com/gcicc/tableGenCompare.git
cd tableGenCompare
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
jupyter notebook STG-Driver-breast-cancer.ipynb
```

### Key Dependencies

| Group | Packages |
|---|---|
| Core | pandas, numpy, scipy |
| ML | scikit-learn >= 1.7.2, xgboost == 2.1.3 |
| Deep Learning | PyTorch |
| Synthetic Data | sdv, ctgan, copulas, GANerAid |
| HPO | optuna >= 4.0.0 |
| Visualization | matplotlib, seaborn, plotly |

---

## 2. Data Preprocessing and EDA

### What This Section Does

Section 2 loads the clinical dataset, runs `run_comprehensive_eda()`, and prepares the data for model training. Steps include:

- Dataset overview and summary statistics
- Missing value analysis and MICE imputation
- Categorical encoding (one-hot or label)
- Target variable distribution and class balance assessment
- Correlation heatmaps and feature distribution plots

## Output Files

| File | Description | How to Interpret |
|---|---|---|
| `column_analysis.csv` | Per-column profile: data type, unique values, missing count/percent, min/max | Use this to verify that the preprocessing pipeline handled each column correctly. Check that missing percentages dropped to 0% after imputation, and that min/max ranges are clinically plausible. |
| `target_analysis.csv` | Class label counts and percentages | Reveals class imbalance. A minority class below 30% may cause generators to underrepresent that class. Record the ratio for later comparison against synthetic target distributions. |
| `target_balance_metrics.csv` | Class balance ratio and imbalance category | A `Class_Balance_Ratio` below 0.5 flags the dataset as "Highly Imbalanced." This warns you that GAN-based generators may struggle with the minority class and that evaluation metrics sensitive to imbalance (F1, balanced accuracy) should be preferred over raw accuracy. |
| `target_correlations.csv` | Pearson correlation of each feature with the target | Identifies the features most predictive of the outcome. Features with high absolute correlation should remain strongly correlated in synthetic data; check this against the Section 3/5 correlation comparison plots. |
| `correlation_matrix.csv` | Full pairwise Pearson correlation matrix | Numerical companion to the heatmap below. Use it for precise values when the heatmap color scale is ambiguous. |

## Graphics

**`correlation_heatmap.png`**

A color-coded matrix of pairwise Pearson correlations across all numeric features. Each cell shows the correlation coefficient between two columns.

**How to read it:** - **Color scale:** Dark red = strong positive correlation (+1), dark blue = strong negative correlation (-1), white = near zero. - **Diagonal:** Always +1 (each feature correlated with itself). - **Look for:** Clusters of warm colors indicating groups of co-varying features. Strong off-diagonal correlations represent relationships that the generative models must preserve. Compare this heatmap to the per-model correlation comparison plots in Sections 3 and 5 to assess how well each model captured these relationships.

**`feature_distributions_part1.png` / `feature_distributions_part2.png`**

Histograms (with KDE overlays) for each numeric feature, split across two pages when the feature count exceeds one page.

**How to read them:** - Each subplot shows the empirical distribution of one numeric column. - **Look for:** Skewness, multimodality, outlier tails. Heavily skewed features (e.g., bilirubin, liver enzymes) are harder

for generators to replicate. Note features with long right tails — these are common in clinical lab values and are often compressed by GANs. - These serve as the "ground truth" against which Section 3/5 distribution comparison plots are judged.

`feature_distributions_categorical.png`

Bar charts showing the frequency of each category within categorical columns (e.g., gender).

**How to read it:** - Bars represent the count or proportion in each category. - **Look for:** Category imbalance. If one category dominates (e.g., 75% male), generators may amplify or reduce this skew. Compare against the contingency similarity metric in Sections 3/5.

### Caveats

1. **No synthetic dataset perfectly replicates real data** — expect differences, especially in tail distributions
2. **Small datasets amplify imperfections** — fewer than 500 rows limits GAN learning
3. **Categorical-heavy data is harder for GANs** — many categories create sparse representations
4. **Class imbalance affects generation quality** — minority classes may be underrepresented
5. **Privacy-utility tradeoff is fundamental** — stronger privacy guarantees reduce fidelity
6. **Stochasticity across runs** — results vary between runs due to random initialization
7. **GAN training instability** — mode collapse and vanishing gradients are inherent risks

---

## 3. Baseline Model Evaluation (Default Parameters)

### What This Section Does

Section 3 trains all 8 generative models using their **default (library-provided) hyperparameters** and runs a comprehensive SDAC evaluation. This establishes a baseline against which optimized models (Section 5) are compared.

### Available Models

| Model | Type | Best Suited For |
|---|---|---|
| **CTGAN** | GAN | General-purpose tabular data with mixed types |
| **CTAB-GAN** | GAN | Datasets needing enhanced preprocessing |
| **CTAB-GAN+** | GAN | Stable training on complex datasets (WGAN-GP) |
| **GANerAid** | GAN | Clinical/healthcare-specific data |
| **CopulaGAN** | Statistical | Preserving column correlations |
| **TVAE** | VAE | Stable training without adversarial dynamics |
| **PATE-GAN** | GAN | Privacy-sensitive data (differential privacy) |

| Model | Type | Best Suited For |
|---|---|---|
| **MEDGAN** | GAN | Discrete medical features |

## A Priori Expectations

- **CTAB-GAN+** expected to outperform CTAB-GAN due to WGAN-GP stability
- **CopulaGAN** strong at correlation preservation (copula-based architecture)
- **TVAE** more stable training than GANs (no adversarial dynamics)
- **GANerAid** may show domain-specific advantages on clinical data
- **PATE-GAN** strongest privacy but lower fidelity (differential privacy noise)

## Output Files

### Cross-Model Summary Files

| File | Description | How to Interpret |
|---|---|---|
| `sdac_evaluation_summary.csv` | One row per model, all SDAC metrics as columns (Privacy, Fidelity, Utility, Fairness, XAI) | The primary comparison table. Sort by any column to rank models on that dimension. Compare Privacy_Score (higher = safer) vs. Fidelity metrics (lower JSD/KS/KL = better). A model with high privacy but poor fidelity is safe but not useful; the reverse is useful but risky. |
| `privacy_summary.csv` | Per-model privacy detail: NNDR mean/std, memorization score/count, re-identification risk, DCR mean | Drill into privacy concerns. NNDR_Std reveals how variable the nearest-neighbor distances are — a low std with a high mean is the ideal (consistently private). Memorized_Count = 0 for all models means no synthetic record is a near-exact copy of a real one. |
| `privacy_dashboard.png` | Multi-panel privacy visualization across all models | See graphic interpretation below. |
| `sdac_radar_chart.png` | Radar (spider) plot with one polygon per model, axes = SDAC dimension composite scores | See graphic interpretation below. |
| `sdac_heatmap.png` | Color-coded grid: rows = models, columns = individual metrics, grouped by SDAC category | See graphic interpretation below. |

### Per-Model Files (one folder per model, e.g., `CTGAN/`, `TVAE/`)

| File | Description | How to Interpret |
|---|---|---|
| `evaluation_summary.csv` | Overall quality score, quality label (Excellent/Good/Fair/Poor), plus sub-scores for statistical similarity, distribution similarity, correlation preservation, PCA similarity, and ML utility | The `Overall_Quality_Score` is a weighted composite. `Quality_Assessment` translates it to a human-readable label. Use the sub-scores to diagnose *where* a model is weak — e.g., high correlation preservation but low distribution similarity means the model captures relationships but distorts marginals. |
| `statistical_similarity.csv` | Per-column comparison: real mean, synthetic mean, mean similarity, std similarity, overall similarity | One row per numeric feature. `mean_similarity` close to 1.0 means the synthetic mean is close to the real mean. Look for columns with low `overall_similarity` — these are the features the model struggled to replicate. Often, features with heavy skew or extreme outliers (e.g., liver enzymes) show the lowest similarity. |
| `correlation_comparison.png` | Side-by-side correlation heatmaps: real (left) vs. synthetic (right) | See graphic interpretation below. |
| `distribution_comparison.png` | Overlaid histograms: real (blue) vs. synthetic (orange) for each numeric feature | See graphic interpretation below. |
| `pca_comparison_with_outcome.png` | PCA scatter plots: real vs. synthetic, colored by target class | See graphic interpretation below. |

## Graphics Interpretation Guide

### `correlation_comparison.png`

Two side-by-side Pearson correlation heatmaps (real on left, synthetic on right) for a single model.

**How to read it:** - Compare the two matrices cell-by-cell. Identical color patterns mean the model perfectly preserved inter-feature relationships. - **Look for:** Color shifts in specific cells — these indicate relationships the generator distorted. For example, if "bilirubin vs. albumin" is dark red in the real heatmap but orange in the synthetic one, the model weakened that correlation. - The overall similarity is quantified by the `Fidelity_Corr_Sim` metric in the SDAC summary (1.0 = perfect match).

### `distribution_comparison.png`

Overlaid histograms for each numeric column, showing real data (typically blue) and synthetic data (typically orange) on the same axes.

**How to read it:** - When the two histograms overlap almost entirely, the model faithfully reproduced that feature's distribution. - **Look for:** Gaps where one distribution extends beyond the other (tail truncation), peaks that are shifted (mode displacement), or bimodal features that became unimodal (mode collapse). - Quantified by `Fidelity_JSD` (Jensen-Shannon Divergence) and `Fidelity_KS` (Kolmogorov-Smirnov statistic) — both should be close to 0.

`pca_comparison_with_outcome.png`

Two PCA scatter plots (real on left, synthetic on right), each point colored by the target class label. The first two principal components are on the x/y axes.

**How to read it:** - The cloud shape, spread, and class separation should look similar between the two panels. - **Look for:** If the synthetic plot shows class clusters merging (losing separation), the model has degraded discriminative structure. If the spread is noticeably tighter, the model is under-generating variance (mode collapse). - This is a holistic, visual check — it complements the numerical metrics by showing multi-dimensional structure in two dimensions.

`sdac_radar_chart.png`

A radar (spider) chart with 5 axes corresponding to the SDAC dimensions (Privacy, Fidelity, Utility, Fairness, XAI). Each model is drawn as a polygon.

**How to read it:** - A larger polygon means better overall performance. A balanced polygon (similar extent on all axes) is preferable to one with extreme spikes and dips. - **Look for:** Models that dominate on most axes. If a model has a deep indentation on one axis (e.g., Privacy), it flags a weakness that may be disqualifying for clinical use. - Each axis is a composite score normalized to [0, 1] so that dimensions with different native scales are comparable.

`sdac_heatmap.png`

A color-coded grid with models as rows and individual SDAC metrics as columns, grouped by dimension.

**How to read it:** - **Color scale:** Green = better, red = worse (for each metric, the direction is normalized so green is always desirable). - **Look for:** Full rows of green (a model excelling everywhere) or red columns (a metric where all models struggle). Clustered red in the Privacy columns may indicate a dataset-level issue (e.g., too-small dataset) rather than a model failure. - This is the most information-dense single output — use it to quickly identify where each model sits relative to the field on every individual metric.

`privacy_dashboard.png`

A multi-panel visualization summarizing privacy across all models, typically including: - Bar charts of DCR and NNDR per model - Memorization score comparison - Re-identification risk comparison

**How to read it:** - Taller bars for DCR and NNDR are better (greater distance from real records). - Memorization and re-identification bars should be at or near zero. - **Look for:** Models with unusually low DCR (synthetic records too close to real ones) or any non-zero memorization score (near-exact copies detected). PATE-GAN and MEDGAN typically show the highest DCR due to their noise-injection and encoding-based architectures, respectively.

---

## 4. Hyperparameter Optimization

## What This Section Does

Section 4 uses **Optuna Bayesian optimization** to search for hyperparameters that maximize a combined quality objective for each model. This is the bridge between the baseline evaluation (Section 3) and the optimized evaluation (Section 5).

## How Optimization Works

- **Engine:** Optuna with TPE (Tree-structured Parzen Estimator) sampler
- **Direction:** Maximize combined score
- **Trial counts:** Smoke test = 5 trials, Full = 50 trials per model
- **Pruning:** `MedianPruner` for CTGAN/CTAB-GAN+ (early-stops unpromising trials)

## Objective Function

```
Combined_Score = 0.6 * Similarity + 0.4 * Accuracy
```

- **Similarity** = Earth Mover's Distance + correlation preservation
- **Accuracy** = TSTR framework with XGBoost classifier

Formally:

$$\text{Combined_Score} = 0.6 \times \left(1 - \overline{W}_d + r_{\text{corr}}\right)/2 \; + \; 0.4 \times \text{Acc}_{\text{TSTR}}$$

where $\overline{W}_d$ is the mean Wasserstein distance across columns (lower = better, so it is inverted), $r_{\text{corr}}$ is the Pearson correlation similarity between the real and synthetic correlation matrices, and $\text{Acc}_{\text{TSTR}}$ is the XGBoost accuracy under the Train-Synthetic-Test-Real scenario.

Source: `src/objective/functions.py`

## Editing the Objective Function

- Change weights: modify lines 24-26 in `src/objective/functions.py`
- Replace classifier: swap the model in the accuracy calculation
- Custom objective: pass `custom_objective_fn` to `optimize_models_batch()`

## The Pilot Phase: Using Smoke Test Mode

The framework supports a **pilot phase** via smoke test mode ( `n_trials=5` ). This is a fast, low-cost way to:

1. **Verify the pipeline runs end-to-end** without waiting hours for full optimization
2. **Spot-check search spaces** — if all 5 trials score similarly, the search space may be too narrow or the objective insensitive to those parameters
3. **Identify models that are fundamentally unsuitable** — a model scoring below 0.40 in 5 trials is unlikely to improve substantially in 50
4. **Estimate wall-clock time** — multiply pilot duration by ~10 to approximate full-mode runtime

### Opportunities to Complement the Pilot

After reviewing pilot results, consider these actions before committing to full optimization:

| Observation | Recommended Action |
|---|---|
| Best score hits the edge of a parameter range | Widen that parameter's search bounds in `src/models/search_spaces.py` |
| One parameter dominates importance across all models | Consider fixing it at its pilot-best value to let Optuna explore other dimensions |
| A model's pilot score is close to its Section 3 default score | The model may be near its ceiling — consider dropping it from full optimization to save compute |
| Two models have nearly identical architectures and scores | Keep only the better-performing variant (e.g., CTAB-GAN+ over CTAB-GAN) |
| Pilot reveals mode collapse (near-zero variance in synthetic data) | Increase minimum training epochs or add regularization before full runs |
| Wall-clock time for pilot exceeds budget for full mode | Switch to a staged optimization approach ( `src/models/staged_optimization.py` ) or reduce the model roster |

## After a Full Trial Run

1. Review `study.best_params` for each model
2. Expand search boundaries if optimal values hit the edge of the range
3. Fix low-importance parameters to reduce search space
4. Compare best scores across models to decide which to carry forward to Section 5

## Output Files

| File | Description | How to Interpret |
|---|---|---|
| `best_parameters.csv` | Every tuned parameter for every model: name, value, type, best score, trial number | The authoritative record of what Section 5 will use. The `best_score` column shows the Combined_Score achieved by the winning trial. The `trial_number` column reveals how early/late in the search the best trial occurred — an early trial (e.g., trial 2 of 50) may suggest the search space is too narrow or the landscape is flat. |
| `best_parameters_summary.csv` | One row per model: best score, trial number, parameter count | Quick comparison table. Rank models by `best_score` to see the optimization hierarchy. Models with very low scores (e.g., < 0.45) may not benefit from further tuning. |

## Interactive Visualizations (Plotly HTML)

Section 4 produces three interactive HTML charts per model. Open them in a browser — they support hover, zoom, and pan.

`optim_history_{Model}.html`

A line/scatter plot showing the objective value for each trial, with a running-best line.

**How to read it:** - **X-axis:** Trial number. **Y-axis:** Combined_Score. - Each point is one trial. The step-line traces the best score found so far. - **Look for:** A curve that plateaus early suggests the search space is well-explored (or too small). A curve still climbing at the final trial suggests more trials would help. Large jumps indicate the optimizer found a promising region.

`param_importance_{Model}.html`

A horizontal bar chart ranking each hyperparameter by its influence on the objective.

**How to read it:** - Longer bars = more important parameters (higher fANOVA importance). - **Look for:** Parameters with near-zero importance can be fixed at any reasonable value in future runs, reducing the search space. The top 2-3 parameters are where tuning budget should be concentrated. - Importance is computed via functional ANOVA (fANOVA), which decomposes variance in the objective attributable to each parameter.

`parallel_coord_{Model}.html`

A parallel coordinates plot where each vertical axis represents a hyperparameter and each line represents a trial, colored by objective value.

**How to read it:** - Lines are colored from blue (low score) to red (high score). - **Look for:** Regions where red lines converge on a narrow band — these are the optimal ranges for that parameter. If red lines span the full range of a parameter, that parameter has little effect. Crossing patterns between adjacent axes reveal interactions.

`optuna_summary_all_models.png`

A static bar chart comparing the best Combined_Score across all models.

**How to read it:** - Taller bars = better optimized models. - **Look for:** The gap between the top model and the rest. A tight cluster at the top means several models are competitive after tuning. A single dominant bar means one model clearly benefits most from hyperparameter optimization.

---

# 5. Optimized Model Evaluation

## What This Section Does

Section 5 **retrains every model using its best hyperparameters** from Section 4, then runs the **full SDAC evaluation** (including MIA and optionally fairness metrics). This is the definitive assessment of each model's quality.

The structure mirrors Section 3 but with two key differences: 1. Models use optimized hyperparameters instead of defaults 2. The evaluation can include MIA (Membership Inference Attack) and fairness metrics

## Running Evaluation

```
# Section 5 — full SDAC evaluation with optimized parameters
results = evaluate_trained_models(
```

```
    section_number=5,
    scope=globals(),
    compute_mia=True,           # Enable Membership Inference Attack
    protected_col="gender"      # Optional: enable fairness metrics
)
```

## Output Files

Section 5 produces the same file types as Section 3. Refer to the Section 3 output descriptions for interpretation guidance. The key files are:

### Cross-Model Summary Files

| File | Description | How to Interpret |
|---|---|---|
| `sdac_evaluation_summary.csv` | Full SDAC metrics for all models using optimized parameters | **Compare directly against the Section 3 version** to measure the impact of hyperparameter tuning. Models whose scores improved substantially validate the optimization. Models that remained flat may already have been near their architectural ceiling. |
| `privacy_summary.csv` | Detailed privacy metrics with optimized parameters | Check whether tuning degraded privacy. If DCR dropped or NNDR decreased relative to Section 3, the optimizer may have pushed models toward memorization in pursuit of higher fidelity. |
| `privacy_dashboard.png` | Multi-panel privacy visualization (same format as Section 3) | Compare side-by-side with the Section 3 dashboard to visualize privacy impact of tuning. |
| `sdac_radar_chart.png` | Radar chart of SDAC composite scores | Polygons should generally be larger than in Section 3. Any dimension that shrank after optimization warrants investigation. |
| `sdac_heatmap.png` | Full metrics heatmap | Look for shifts from red to green (improvement) or green to red (regression) compared to Section 3. |

### Per-Model Files (same structure as Section 3)

| File | Description |
|---|---|
| `{Model}/evaluation_summary.csv` | Overall quality score and sub-scores for the optimized model |
| `{Model}/statistical_similarity.csv` | Per-column real vs. synthetic mean/std comparison |
| `{Model}/correlation_comparison.png` | Side-by-side correlation heatmaps |
| `{Model}/distribution_comparison.png` | Overlaid real vs. synthetic histograms |
| `{Model}/pca_comparison_with_outcome.png` | PCA scatter by target class |

All graphics follow the same interpretation as described in Section 3. The key question when reviewing Section 5 outputs is: **did optimization improve the model, and at what cost to privacy?**

### How the Best Model Is Chosen

1. **Primary:** Model with highest `Combined_Score` (0.6 x Similarity + 0.4 x Accuracy)
2. **Secondary tiebreaker:** Privacy score

### Section 3 vs. Section 5: What to Compare

| Aspect | Section 3 (Baseline) | Section 5 (Optimized) |
|---|---|---|
| Hyperparameters | Library defaults | Optuna-tuned best parameters |
| Purpose | Establish baseline; identify architectural strengths/weaknesses | Final model selection and deployment readiness |
| MIA | Typically disabled | Typically enabled |
| Fairness | Typically disabled | Enabled if `protected_col` specified |
| Expected result | Models ranked by inherent architecture quality | Models ranked by tuned performance |

---

## 6. Generating Synthetic Samples

### In-Session Generation

```
# Access trained model from evaluation results
model = results['CTGAN']['model']
synthetic_data = model.generate(n_samples=1000)
synthetic_data.to_csv("my_synthetic_data.csv", index=False)
```

### Retraining from Saved Parameters

```
import pandas as pd
from src.models.model_factory import ModelFactory

# Load best parameters
params = pd.read_csv("results/dataset/best_parameters.csv")
best_params = params.iloc[0].to_dict()

# Recreate and train
model = ModelFactory.create("CTGAN", **best_params)
model.train(real_data, target_column="diagnosis")
synthetic = model.generate(n_samples=500)
```

---

# 7. SDAC Metrics Reference

All metrics are organized by the 5 SDAC dimensions. Each metric includes a plain-language definition, mathematical/statistical definition, range, and interpretation guide.

---

## 7.1 Privacy Metrics

Privacy metrics measure how well synthetic data protects the confidentiality of real patient records.

### DCR (Distance to Closest Record)

| | |
|---|---|
| **What it measures** | The average distance between each synthetic record and the nearest real record |
| **Plain language** | How far away is each fake record from the closest real patient? |
| **Mathematical definition** | For each synthetic record $\mathbf{s}_i$, compute the minimum Euclidean distance to any real record: $\mathrm{DCR}_i = \min_j \lVert \mathbf{s}_i - \mathbf{r}_j \rVert_2$. The reported value is the mean: $\mathrm{DCR} = \frac{1}{n}\sum_{i=1}^{n} \mathrm{DCR}_i$, where $n$ is the number of synthetic records and all features are standardized before distance computation. |
| **Range** | 0 to +∞ (higher = more private) |
| **Interpretation** | A value near 0 means synthetic records are nearly identical to real ones (bad). Higher values mean synthetic records are sufficiently different from any real patient. |
| **Source** | `src/evaluation/privacy.py` |

### NNDR (Nearest Neighbor Distance Ratio)

| | |
|---|---|
| **What it measures** | Ratio of each synthetic record's distance to the nearest real record vs. its distance to the nearest other synthetic record |
| **Plain language** | Is each fake record closer to a real patient or to other fake records? |
| **Mathematical definition** | For each synthetic record $\mathbf{s}_i$: $\mathrm{NNDR}_i = \frac{\min_j \lVert \mathbf{s}_i - \mathbf{r}_j \rVert_2}{\min_{k \neq i} \lVert \mathbf{s}_i - \mathbf{s}_k \rVert_2}$. The reported value is the mean across all synthetic records. |
| **Range** | 0 to +∞ (values > 1.0 are better) |
| **Interpretation** | NNDR < 1.0 means synthetic records are closer to real data than to other synthetic data — a sign of memorization. NNDR > 1.0 means synthetic records cluster with each other, not with real data. |
| **Source** | `src/evaluation/privacy.py` |

### IMS (Individual Memorization Score)

| | |
|---|---|
| **What it measures** | Percentage of synthetic records that are near-exact copies of real records |
| **Plain language** | What fraction of fake records are essentially copies of real patients? |
| **Mathematical definition** | $\mathrm{IMS} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}[\mathrm{DCR}_i < \tau]$, where $\tau$ is a threshold (typically the 5th percentile of pairwise distances within the real dataset). A synthetic record with $\mathrm{DCR}_i < \tau$ is flagged as memorized. |
| **Range** | 0 to 1 (lower = more private) |
| **Interpretation** | 0% = no memorization (ideal). Values above 5% indicate the model is copying real records rather than learning general patterns. |
| **Source** | `src/evaluation/privacy.py` — reported as `memorization_score` |

## Re-identification Risk

| | |
|---|---|
| **What it measures** | Percentage of synthetic records close enough to a real record that someone could potentially link them back to a real patient |
| **Plain language** | Could someone look at a fake record and figure out which real patient it came from? |
| **Mathematical definition** | $\mathrm{ReID} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\left[\mathrm{DCR}_i < \delta \ \wedge \ \mathrm{unique_match}(\mathbf{s}_i)\right]$, where $\delta$ is a re-identification distance threshold and $\mathrm{unique_match}$ checks whether the closest real record is uniquely closest (no ties). |
| **Range** | 0 to 1 (lower = more private) |
| **Interpretation** | 0% = no re-identification risk. Values above 10% are concerning for patient privacy. |
| **Source** | `src/evaluation/privacy.py` |

## Privacy Score (Composite)

| | |
|---|---|
| **What it measures** | A single composite privacy score combining DCR, NNDR, and memorization |
| **Mathematical definition** | $\mathrm{Privacy_Score} = w_1 \cdot \mathrm{norm}(\mathrm{DCR}) + w_2 \cdot \mathrm{norm}(\mathrm{NNDR}) + w_3 \cdot (1 - \mathrm{IMS})$, where $\mathrm{norm}(\cdot)$ applies min-max normalization across models and the weights sum to 1. |
| **Range** | 0 to 1 (higher = more private) |
| **Interpretation** | 1.0 = maximum privacy (e.g., PATE-GAN and MEDGAN often achieve this due to high noise / encoding). Values below 0.75 warrant closer inspection of DCR and NNDR. |
| **Source** | `src/evaluation/privacy.py` |

## MIA AUC (Membership Inference Attack AUC)

| | |
|---|---|
| **What it measures** | How well a classifier can distinguish whether a given record was in the original training set |

| | |
|---|---|
| **Plain language** | Can an attacker tell whether a specific person's data was used to train the model? |
| **Mathematical definition** | A shadow-model MIA is performed: (1) train a generative model on a subset of real data, (2) label records as "member" (in training set) or "non-member," (3) train a Random Forest attack classifier on features derived from distance-to-synthetic-data. The reported value is the AUC of this attack classifier: $\mathrm{MIA\backslash_AUC} = \int_0^1 \mathrm{TPR}(t)\, d\mathrm{FPR}(t)$, where TPR and FPR are the true/false positive rates at threshold $t$. |
| **Range** | 0.5 to 1.0 (closer to 0.5 = more private) |
| **Interpretation** | AUC = 0.5 means the attacker cannot distinguish members from non-members (ideal). AUC > 0.7 means significant membership leakage. AUC > 0.9 means severe privacy risk. |
| **Note** | Only computed when `compute_mia=True` (Section 5 full evaluation). |
| **Reference** | Shokri et al., "Membership Inference Attacks Against Machine Learning Models" (2017) |
| **Source** | `src/evaluation/privacy.py` |

---

## 7.2 Fidelity Metrics

Fidelity metrics measure how closely the synthetic data matches the statistical properties of the real data.

### JSD (Jensen-Shannon Divergence)

| | |
|---|---|
| **What it measures** | Symmetric measure of difference between the real and synthetic distributions for each column |
| **Plain language** | How different are the shapes of the real vs. synthetic data distributions? |
| **Mathematical definition** | For two distributions $P$ (real) and $Q$ (synthetic), let $M = \frac{1}{2}(P+Q)$. Then: $\mathrm{JSD}(P\|Q) = \frac{1}{2}D_{\mathrm{KL}}(P\|M) + \frac{1}{2}D_{\mathrm{KL}}(Q\|M)$, where $D_{\mathrm{KL}}$ is the Kullback-Leibler divergence. The reported value is the mean JSD across all numeric columns. |
| **Range** | 0 to 1 (lower = more similar) when computed with log base 2 |
| **Interpretation** | 0 = identical distributions. < 0.1 = excellent fidelity. > 0.3 = distributions differ substantially. |
| **Source** | `src/evaluation/sdac_metrics.py` |

### KS Statistic (Kolmogorov-Smirnov)

| | |
|---|---|
| **What it measures** | Maximum difference between the cumulative distribution functions of real and synthetic data |
| **Plain language** | What is the biggest gap between the real and synthetic data when you line them up from smallest to largest? |
| **Mathematical definition** | $\mathrm{KS} = \sup_x \lvert F_{\mathrm{real}}(x) - F_{\mathrm{synth}}(x)\rvert$, where $F$ denotes the empirical cumulative distribution function. The reported value is the mean KS statistic across all numeric columns. |

| | |
|---|---|
| **Range** | 0 to 1 (lower = more similar) |
| **Interpretation** | 0 = identical CDFs. < 0.1 = excellent match. > 0.3 = significant distributional shift. |
| **Reference** | `scipy.stats.ks_2samp` |
| **Source** | `src/evaluation/fidelity.py` |

## KL Divergence (Kullback-Leibler)

| | |
|---|---|
| **What it measures** | Information lost when the synthetic distribution is used to approximate the real distribution |
| **Plain language** | How much information do you lose by using the fake data instead of the real data? |
| **Mathematical definition** | $D_{\mathrm{KL}}(P\|Q) = \sum_x P(x) \ln \frac{P(x)}{Q(x)}$, where $P$ is the real distribution and $Q$ is the synthetic distribution, both discretized via histogram binning. Laplace smoothing ($+\epsilon$) is applied to avoid division by zero. The reported value is the mean across all numeric columns. |
| **Range** | 0 to +∞ (lower = more similar) |
| **Interpretation** | 0 = identical distributions. < 0.1 = excellent. Unlike JSD, KL divergence is asymmetric and unbounded — large values can occur when the synthetic data has zero density where the real data has mass. |
| **Source** | `src/evaluation/fidelity.py` |

## Wasserstein Distance (Earth Mover's Distance)

| | |
|---|---|
| **What it measures** | Minimum "work" needed to transform one distribution into another |
| **Plain language** | How much effort would it take to reshape the fake data to look exactly like the real data? |
| **Mathematical definition** | $W_1(P,Q) = \inf_{\gamma \in \Gamma(P,Q)} \int \|x-y\| \, d\gamma(x,y)$, where $\Gamma(P,Q)$ is the set of all joint distributions with marginals $P$ and $Q$. For one-dimensional distributions this simplifies to: $W_1 = \int_0^1 \|F_P^{-1}(u) - F_Q^{-1}(u)\| \, du$. The reported value is the mean across all numeric columns (after standardization). |
| **Range** | 0 to +∞ (lower = more similar) |
| **Interpretation** | 0 = identical distributions. Sensitive to both shape and location differences. More robust than KL divergence for distributions with non-overlapping support. |
| **Reference** | `scipy.stats.wasserstein_distance` |
| **Source** | `src/evaluation/fidelity.py` |

## Detection AUC

| | |
|---|---|
| **What it measures** | How well a logistic regression classifier can tell real records from synthetic records |
| **Plain language** | If you mix real and fake records together, can a simple model tell them apart? |
| **Mathematical definition** | Real and synthetic records are labeled 0/1, combined, and split into train/test sets. A logistic regression classifier is trained, and the AUC of its ROC curve on the test set is reported: $\text{Detection_AUC} = \int_0^1 \text{TPR}(t)\, d\text{FPR}(t)$. |
| **Range** | 0.5 to 1.0 (closer to 0.5 = better fidelity) |
| **Interpretation** | AUC = 0.5 means the classifier cannot distinguish real from synthetic (excellent fidelity). AUC > 0.8 means synthetic data is easily distinguishable (poor fidelity). |
| **Source** | `src/evaluation/fidelity.py` |

## Correlation Similarity

| | |
|---|---|
| **What it measures** | How well the synthetic data preserves the pairwise correlation structure between columns |
| **Plain language** | If column A and column B are correlated in the real data, are they still correlated the same way in the synthetic data? |
| **Mathematical definition** | Let $C_{\text{real}}$ and $C_{\text{synth}}$ be the Pearson correlation matrices. The similarity is: $\text{Corr_Sim} = 1 - \frac{\lVert C_{\text{real}} - C_{\text{synth}} \rVert_F}{\lVert C_{\text{real}} \rVert_F + \lVert C_{\text{synth}} \rVert_F}$, where $\lVert \cdot \rVert_F$ is the Frobenius norm. |
| **Range** | 0 to 1 (higher = better) |
| **Interpretation** | 1.0 = perfect correlation preservation. < 0.7 = some relationships between variables are being distorted. |
| **Source** | `src/evaluation/sdac_metrics.py` |

## Contingency Similarity

| | |
|---|---|
| **What it measures** | For categorical columns, how well the synthetic data preserves the joint frequency distributions |
| **Plain language** | Are combinations of categorical values (e.g., "Male" + "Diabetic") equally common in real and synthetic data? |
| **Mathematical definition** | For each pair of categorical columns, compute the contingency table (cross-tabulation) for both real and synthetic data. The similarity is the mean of element-wise comparisons: $\text{Cont_Sim} = 1 - \frac{1}{k} \sum_{j=1}^{k} \frac{\lVert T_{\text{real}}^{(j)} - T_{\text{synth}}^{(j)} \rVert_1}{\lVert T_{\text{real}}^{(j)} \rVert_1 + \lVert T_{\text{synth}}^{(j)} \rVert_1}$, where $T^{(j)}$ is the flattened, normalized contingency table for pair $j$ and $k$ is the number of categorical column pairs. |
| **Range** | 0 to 1 (higher = better) |
| **Interpretation** | 1.0 = identical category co-occurrence patterns. Low values indicate the model is not capturing interactions between categorical variables. |

| | |
|---|---|
| **Source** | `src/evaluation/fidelity.py` |

---

## 7.3 Utility Metrics

Utility metrics measure whether synthetic data is useful as a substitute for real data in downstream machine learning tasks.

### TSTR Accuracy (Train Synthetic, Test Real)

| | |
|---|---|
| **What it measures** | Classification accuracy when a model is trained on synthetic data and tested on real data |
| **Plain language** | If you train a model using only fake data, how well does it perform on real patients? |
| **Mathematical definition** | $\mathrm{Acc}_{\mathrm{TSTR}} = \frac{1}{n_{\mathrm{test}}} \sum_{i=1}^{n_{\mathrm{test}}} \mathbf{1}[\hat{y}_i = y_i]$, where $\hat{y}_i$ is the prediction of a classifier trained entirely on synthetic data and $y_i$ is the true label from the real test set. |
| **Range** | 0 to 1 (higher = better) |
| **Interpretation** | Close to TRTR accuracy = synthetic data is a good substitute. Large gap = synthetic data is missing important patterns. Reported for XGBoost (primary), RF, and LR. |
| **Source** | `src/evaluation/trts.py` , `src/evaluation/sdac_metrics.py` |

### TSTR F1 Score

| | |
|---|---|
| **What it measures** | Harmonic mean of precision and recall for the TSTR scenario |
| **Plain language** | Balanced measure of how well a fake-data-trained model identifies positive cases in real patients |
| **Mathematical definition** | $F_1 = 2 \cdot \frac{\mathrm{Precision} \cdot \mathrm{Recall}}{\mathrm{Precision} + \mathrm{Recall}} = \frac{2\mathrm{TP}}{2\mathrm{TP} + \mathrm{FP} + \mathrm{FN}}$, where TP, FP, FN are computed on the real test set using a classifier trained on synthetic data. For multiclass, the weighted average across classes is reported. |
| **Range** | 0 to 1 (higher = better) |
| **Interpretation** | More informative than accuracy when classes are imbalanced. Reported for XGBoost (primary). |
| **Source** | `src/evaluation/sdac_metrics.py` |

### TSTR AUROC

| | |
|---|---|
| **What it measures** | Area under the ROC curve for the TSTR scenario |
| **Plain language** | Overall ability of a fake-data-trained model to rank real patients correctly |

| | |
|---|---|
| **Mathematical definition** | $\mathrm{AUROC} = \int_0^1 \mathrm{TPR}(t)\, d\mathrm{FPR}(t) = P(\hat{p}_{\mathrm{pos}} > \hat{p}_{\mathrm{neg}})$, where $\hat{p}$ is the predicted probability from a classifier trained on synthetic data. Equivalently, it is the probability that a randomly chosen positive real sample is ranked higher than a randomly chosen negative real sample. |
| **Range** | 0.5 to 1.0 (higher = better) |
| **Interpretation** | 0.5 = random guessing. > 0.8 = good. > 0.9 = excellent. Threshold-independent measure. |
| **Source** | `src/evaluation/sdac_metrics.py` |

### ML Efficacy Score

| | |
|---|---|
| **What it measures** | Ratio of TSTR accuracy to TRTR accuracy |
| **Plain language** | What fraction of real-data performance is preserved when training on synthetic data? |
| **Mathematical definition** | $\mathrm{ML_Efficacy} = \frac{\mathrm{Acc}_{\mathrm{TSTR}}}{\mathrm{Acc}_{\mathrm{TRTR}}}$, where $\mathrm{Acc}_{\mathrm{TRTR}}$ is the accuracy of a classifier trained and tested on real data (the ceiling). |
| **Range** | 0 to ~1.0 (higher = better) |
| **Interpretation** | 1.0 = synthetic data is as useful as real data for ML. 0.8 = 80% of performance preserved. Values > 1.0 can occur but usually indicate overfitting or favorable random splits. |
| **Source** | `src/evaluation/sdac_metrics.py` |

### SRA (Synthetic Ranking Agreement)

| | |
|---|---|
| **What it measures** | Whether different ML models rank in the same order when trained on synthetic vs. real data |
| **Plain language** | If Model A beats Model B on real data, does it also beat Model B on synthetic data? |
| **Mathematical definition** | Let $\mathbf{r}_{\mathrm{real}} = (\mathrm{rank}_{\mathrm{RF}}, \mathrm{rank}_{\mathrm{LR}}, \mathrm{rank}_{\mathrm{XGB}})_{\mathrm{TRTR}}$ and $\mathbf{r}_{\mathrm{synth}}$ be the corresponding ranks under TSTR. Then: $\mathrm{SRA} = \rho(\mathbf{r}_{\mathrm{real}}, \mathbf{r}_{\mathrm{synth}})$, where $\rho$ is the Spearman rank correlation coefficient: $\rho = 1 - \frac{6\sum d_i^2}{n(n^2-1)}$ and $d_i$ is the rank difference for classifier $i$. |
| **Range** | -1 to 1 (higher = better) |
| **Interpretation** | 1.0 = identical model rankings. 0 = no agreement. Negative values indicate reversed rankings. With only 3 classifiers, SRA tends to extreme values (-1 or +1). |
| **Source** | `src/evaluation/sdac_metrics.py` |

---

## 7.4 Fairness Metrics

Fairness metrics measure whether the synthetic data introduces or amplifies biases with respect to a protected attribute (e.g., gender, race, age group). These metrics require specifying `protected_col` — they are blank/NaN if no protected attribute is provided.

## Demographic Parity Difference

| | |
|---|---|
| **What it measures** | Difference in positive prediction rates between protected groups |
| **Plain language** | Are different demographic groups predicted positive at the same rate? |
| **Mathematical definition** | $\mathrm{DPD} = \lvert P(\hat{Y}=1 \mid A=0) - P(\hat{Y}=1 \mid A=1)\rvert$, where $A$ is the binary protected attribute and $\hat{Y}$ is the predicted label. Computed using a classifier trained on synthetic data and evaluated on real data. |
| **Range** | 0 to 1 (lower = more fair) |
| **Interpretation** | 0 = perfectly equal prediction rates across groups. Values > 0.1 indicate meaningful disparity. Comparing DPD on real vs. synthetic data reveals whether the model amplifies bias. |
| **Source** | `src/evaluation/fairness.py` |

## Equalized Odds Difference

| | |
|---|---|
| **What it measures** | Maximum difference in true positive rate and false positive rate between protected groups |
| **Plain language** | Do different demographic groups experience the same error rates? |
| **Mathematical definition** | $\mathrm{EOD} = \max\big(\lvert\mathrm{TPR}_{A=0} - \mathrm{TPR}_{A=1}\rvert,\ \lvert\mathrm{FPR}_{A=0} - \mathrm{FPR}_{A=1}\rvert\big)$, where $\mathrm{TPR}_A$ and $\mathrm{FPR}_A$ are the true/false positive rates for group $A$. |
| **Range** | 0 to 1 (lower = more fair) |
| **Interpretation** | 0 = identical error rates across groups. High values mean the model is more accurate for one group than another. |
| **Source** | `src/evaluation/fairness.py` |

## Disparate Impact Ratio

| | |
|---|---|
| **What it measures** | Ratio of positive prediction rates between the disadvantaged and advantaged groups |
| **Plain language** | Is one demographic group proportionally less likely to receive a positive prediction? |
| **Mathematical definition** | $\mathrm{DI} = \frac{P(\hat{Y}=1 \mid A=0)}{P(\hat{Y}=1 \mid A=1)}$, where $A=0$ is conventionally the disadvantaged group. If the denominator is zero, DI is undefined. |
| **Range** | 0 to +∞ (closer to 1.0 = more fair) |
| **Interpretation** | 1.0 = perfect parity. The "four-fifths rule" considers values below 0.8 as evidence of disparate impact. Values > 1.0 indicate the minority group is predicted positive more often. |
| **Source** | `src/evaluation/fairness.py` |

## 7.5 XAI (Explainability) Metrics

XAI metrics measure whether the synthetic data preserves the explainability properties of models trained on real data.

### Feature Importance Correlation

| | |
|---|---|
| **What it measures** | Pearson correlation between Random Forest feature importances trained on real vs. synthetic data |
| **Plain language** | Do the same features matter for prediction in both real and synthetic data? |
| **Mathematical definition** | Let $\mathbf{f}_{\text{real}}$ and $\mathbf{f}_{\text{synth}}$ be the vectors of Gini-importance values from Random Forests trained on real and synthetic data, respectively. Then: $\text{FI}\backslash_\text{Corr} = \frac{\text{Cov}(\mathbf{f}_{\text{real}}, \mathbf{f}_{\text{synth}})}{\sigma_{\mathbf{f}_{\text{real}}} \cdot \sigma_{\mathbf{f}_{\text{synth}}}}$ (Pearson correlation coefficient). |
| **Range** | -1 to 1 (higher = better) |
| **Interpretation** | 1.0 = identical feature importance rankings. > 0.8 = excellent preservation. Low values mean the synthetic data has shifted which features are most predictive. |
| **Source** | `src/evaluation/xai_metrics.py` |

### SHAP Distance

| | |
|---|---|
| **What it measures** | Cosine distance between mean SHAP value vectors from models trained on real vs. synthetic data |
| **Plain language** | Do individual predictions get explained the same way when using real vs. synthetic training data? |
| **Mathematical definition** | Let $\phi_{\text{real}}$ and $\phi_{\text{synth}}$ be the mean absolute SHAP value vectors. The SHAP distance is the cosine distance: $\text{SHAP}\backslash_\text{Dist} = 1 - \frac{\phi_{\text{real}} \cdot \phi_{\text{synth}}}{\lVert\phi_{\text{real}}\rVert \cdot \lVert\phi_{\text{synth}}\rVert}$. |
| **Range** | 0 to 1 (lower = better) |
| **Interpretation** | 0 = identical explanations. < 0.2 = good preservation. High values mean the model's reasoning changes when trained on synthetic data. |
| **Note** | Requires the `shap` library. Returns NaN if SHAP is not installed. |
| **Reference** | Lundberg & Lee, "A Unified Approach to Interpreting Model Predictions" (NeurIPS 2017) |
| **Source** | `src/evaluation/xai_metrics.py` |

---

## 7.6 TRTS Framework Detail

The TRTS (Train Real/Test Synthetic) framework evaluates synthetic data through 4 classification scenarios:

| Scenario | Train On | Test On | What It Shows |
|---|---|---|---|
| **TRTR** | Real | Real | Baseline (best possible performance) |
| **TSTR** | Synthetic | Real | Can synthetic data replace real data for training? |
| **TRTS** | Real | Synthetic | Does the model generalize to synthetic data? |
| **TSTS** | Synthetic | Synthetic | Internal consistency of synthetic data |

Each scenario reports 30+ classification metrics. The **primary classifier** is XGBoost; Random Forest and Logistic Regression run as secondary classifiers (results stored under `*_RF` and `*_LR` keys).

Key metrics per scenario: accuracy, balanced accuracy, precision, recall, F1, specificity, MCC, Cohen's Kappa, AUROC, AUPRC, Brier Score.

**How to use the 4 scenarios together:** - **TRTR** sets the ceiling. No synthetic-data scenario should consistently exceed it. - **TSTR** is the most important for utility — it directly answers "can I train on synthetic and deploy on real?" - **TRTS** reveals whether real-data-trained models transfer to synthetic data. A large gap between TRTR and TRTS suggests the synthetic data occupies a different region of feature space. - **TSTS** measures internal consistency. If TSTS >> TSTR, the synthetic data may be too self-similar (mode collapse).

---

### 7.7 Quality Score Labels

| Score Range | Label |
|---|---|
| >= 0.80 | Excellent |
| 0.60 - 0.79 | Good |
| 0.40 - 0.59 | Fair |
| < 0.40 | Poor |

---

## 8. Architecture Reference

### Directory Structure

```
tableGenCompare/
├── setup.py                    # Backward-compatible re-export layer
├── requirements.txt            # Python dependencies
├── STG-Driver-*.ipynb          # Workflow notebooks (one per dataset)
│
├── src/
│   ├── config.py               # Session management and defaults
│   ├── models/
│   │   ├── model_factory.py    # ModelFactory.create()
│   │   ├── search_spaces.py    # Hyperparameter search spaces
│   │   ├── batch_training.py   # Batch training and retraining
```

```
│   │   ├── batch_optimization.py  # Optuna batch HPO
│   │   └── implementations/    # Per-model wrappers
│   ├── data/
│   │   ├── preprocessing.py    # MICE imputation, encoding
│   │   ├── eda.py              # run_comprehensive_eda()
│   │   └── summary.py          # Data summaries
│   ├── evaluation/
│   │   ├── sdac_metrics.py     # Unified SDAC orchestrator
│   │   ├── quality.py          # JSD, correlation, ML utility
│   │   ├── fidelity.py         # KS, KL, WD, Detection AUC
│   │   ├── trts.py             # 4-scenario TRTS (XGBoost primary)
│   │   ├── privacy.py          # DCR, NNDR, MIA, memorization
│   │   ├── fairness.py         # Demographic parity, equalized odds
│   │   ├── xai_metrics.py      # Feature importance, SHAP distance
│   │   └── batch.py            # Batch evaluation pipeline
│   ├── objective/
│   │   └── functions.py        # Optuna objective function
│   ├── visualization/
│   │   ├── section2.py         # Correlation heatmap, feature distributions
│   │   ├── section3.py         # Per-model comparison plots
│   │   ├── section4.py         # Optuna visualizations
│   │   └── section5.py         # TRTS, privacy, SDAC charts
│   └── utils/
│       ├── paths.py            # Path management
│       └── parameters.py       # Parameter save/load
│
├── data/                       # Input datasets
├── results/                    # Generated outputs (per dataset, per section)
├── docs/                       # Documentation
│
├── CTAB-GAN/                   # Git submodule
├── CTAB-GAN-Plus/              # Git submodule
└── GANerAid/                   # Git submodule
```

## Results Directory Structure

```
results/{dataset-name}/{date}/
├── Section-2/
│   ├── column_analysis.csv           # Per-column data profile
│   ├── target_analysis.csv           # Target class distribution
│   ├── target_balance_metrics.csv    # Class balance ratio
│   ├── target_correlations.csv       # Feature-target correlations
│   ├── correlation_matrix.csv        # Full pairwise correlations
│   ├── correlation_heatmap.png       # Visual correlation matrix
│   ├── feature_distributions_part1.png  # Numeric feature histograms
│   ├── feature_distributions_part2.png  # (continued)
│   └── feature_distributions_categorical.png  # Categorical bar charts
├── Section-3/
│   ├── sdac_evaluation_summary.csv   # Cross-model SDAC metrics (baseline)
│   ├── privacy_summary.csv           # Detailed privacy breakdown
│   ├── sdac_radar_chart.png          # SDAC dimension radar chart
│   ├── sdac_heatmap.png              # Full metrics heatmap
│   ├── privacy_dashboard.png         # Multi-panel privacy visualization
│   └── {MODEL}/                      # Per-model folder (×7-8 models)
│       ├── evaluation_summary.csv    # Quality score and sub-scores
```

```
│       ├── statistical_similarity.csv # Per-column real vs. synthetic
│       ├── correlation_comparison.png # Side-by-side correlation heatmaps
│       ├── distribution_comparison.png # Overlaid histograms
│       └── pca_comparison_with_outcome.png # PCA scatter by class
├── Section-4/
│   ├── best_parameters.csv           # All tuned parameters per model
│   ├── best_parameters_summary.csv   # One-row-per-model summary
│   ├── optuna_summary_all_models.png # Best score bar chart
│   ├── optim_history_{Model}.html    # Trial-by-trial objective plot (×7)
│   ├── param_importance_{Model}.html # Parameter importance ranking (×7)
│   └── parallel_coord_{Model}.html   # Parallel coordinates (×7)
└── Section-5/
    ├── sdac_evaluation_summary.csv   # Cross-model SDAC metrics (optimized)
    ├── privacy_summary.csv           # Detailed privacy breakdown
    ├── sdac_radar_chart.png          # SDAC dimension radar chart
    ├── sdac_heatmap.png              # Full metrics heatmap
    ├── privacy_dashboard.png         # Multi-panel privacy visualization
    └── {MODEL}/                      # Per-model folder (×7-8 models)
        ├── evaluation_summary.csv
        ├── statistical_similarity.csv
        ├── correlation_comparison.png
        ├── distribution_comparison.png
        └── pca_comparison_with_outcome.png
```

---

**Document Version:** 2.0 **Last Updated:** March 16, 2026 **Framework Version:** 7.0 (SDAC Evaluation Framework)